

Generative Models for Cat Face Synthesis: A Comparative Study of Variational Autoencoders, Generative Adversarial Networks, and Diffusion Models

Pawel Pozorski¹, Pawel Florek¹

¹ Warsaw University of Technology, Poland

{pawel.pozorski, pawel.florek}.stud@pw.edu.pl

Abstract

This report presents a comparative study of three major generative modeling paradigms—Variational Autoencoders (VAE), Generative Adversarial Networks (GAN), and Diffusion Models—applied to the synthesis of 128×128 cat face images. We evaluate β -VAE and VQ-VAE-1 (VAE family), WGAN-GP and SN-GAN (GAN family), DDIM and a Tiny Latent Diffusion Model (diffusion family), and add a classical PixelCNN autoregressive prior over VQ-VAE code indices as a baseline for latent generative modeling. The study employs structured grid search over key hyperparameters. We evaluate models using Fréchet Inception Distance (FID), latent interpolations, and a qualitative comparison of PixelCNN versus Tiny LDM (sample quality and wall-clock generation time). Furthermore, we conduct an exploratory *chimera* extension: WGAN-GP trained on $\sim 25,000$ mixed dogs-and-cats images at 64×64 to observe distribution averaging phenomena. This work provides a practical framework for comparing generative paradigms.

Index Terms: generative models, variational autoencoders, generative adversarial networks, diffusion models, PixelCNN, latent diffusion, image synthesis, cat generation, FID, hyperparameter optimization, mode collapse

Contents

1	Introduction	2
2	Methodology	2
2.1	Datasets and Preprocessing	2
2.2	Generative Model Architectures	2
2.3	Training Setup and Hyperparameter Grid	3
2.4	Evaluation Metrics	4
3	Results	4
3.1	Fréchet Inception Distance	4
3.2	Qualitative Samples	4
3.3	Latent Interpolation	4
3.4	PixelCNN vs. Tiny LDM	4
3.5	Chimera Extension	4
4	Conclusions	4
5	AI Disclosure	4
6	References	4
A	Additional Experimental Details	5
B	Code Availability	5

1. Introduction

Generative modeling of images has been a central research challenge in machine learning, with three primary paradigms emerging over the past decade: Variational Autoencoders (VAEs) [1], Generative Adversarial Networks (GANs) [2], and Diffusion Models [3]. Each paradigm offers distinct theoretical foundations, training dynamics, and trade-offs in quality, diversity, and training stability. VAEs provide a principled probabilistic framework but often produce blurrier samples; GANs generate sharp images but suffer from training instability and mode collapse; diffusion models have recently demonstrated superior sample quality but require longer inference times.

The selection of appropriate generative models for practical applications remains highly dependent on specific constraints: available computational resources, target sample quality, required inference speed, and stability during training. In this project, we conduct a systematic comparison of representative architectures across all three paradigms on a well-defined task: generating cat face images from the Kaggle Cat Dataset.

Our objectives align with the specific project requirements: (i) standardize varying image resolutions utilizing raw annotation data to accommodate limited computing resources; (ii) systematically compare generation quality across three paradigms (VAE, GAN, Diffusion) using quantitative metrics (FID) and qualitative assessment; (iii) directly address mode collapse through stabilized GAN architectures (WGAN-GP with gradient penalty, SN-GAN with spectral normalization); (iv) analyze latent space smoothness via interpolation: generating 10-image sequences (2 random endpoints + 8 linear intermediates) to assess semantic continuity; and (v) explore cross-species generalization via a *chimera* experiment: WGAN-GP on mixed "Dogs vs. Cats" data at 64×64 to investigate whether the unconditional model learns to separate classes or exhibits distribution averaging (e.g., chimera-like outputs).

2. Methodology

2.1. Datasets and Preprocessing

The primary dataset consists of over 9,000 cat images sourced from the Kaggle Cat Dataset [4]. Raw images exhibit significant variability in resolution, aspect ratio, and face positioning, which poses a challenge for batch processing and limited computing resources. To address this, we apply the following preprocessing pipeline:

1. **Annotation Parsing:** Instead of relying on pre-trained face detectors, we extract the exact 9 coordinate points (eyes, mouth, ears) provided in the corresponding `.cat` annotation files.
2. **Face Cropping:** Bounding boxes are dynamically calculated based on the extreme coordinates of the facial landmarks with a 35% pad on the landmark extent (`BBOX_PAD_RATIO=0.35`).
3. **Resizing:** Cropped regions are standardized to 128×128 pixels using Lanczos resampling (`PIL.Image.LANCZOS`), drastically reducing VRAM footprint and training time.
4. **Normalization:** All cat-face training tensors are mapped to $[-1, 1]$ via `ToTensor` and per-channel normalization with mean 0.5 and std 0.5, matching tanh decoder/generator outputs; pixel DDIM sampling clamps predicted images to $[-1, 1]$ each step.
5. **Training augmentation:** The cat-face train split applies `RandomHorizontalFlip` ($p=0.5$); validation tensors

are not augmented.

Processed images are written to `train.npy` and `val.npy` using a 90/10 train/validation split with a fixed assignment seed (42).

Comparison of raw vs. processed images is shown in Figure 1.

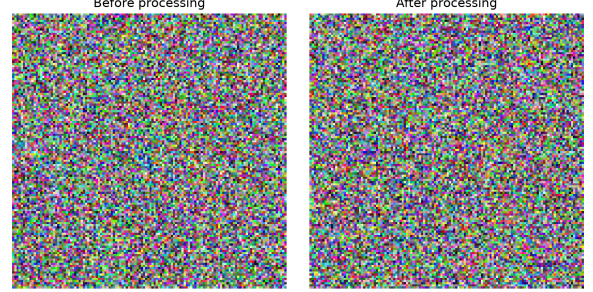


Figure 1: Sample images after preprocessing: cropped and resized to 128×128 pixels.

For the exploratory chimera extension, we utilize the Kaggle "Dogs vs. Cats" competition benchmark dataset [5], which contains $\sim 25,000$ mixed images (Figure 2). Since this dataset lacks facial landmark annotations, applying the coordinate-based cropping procedure is impossible. Instead, we apply the same scale-and-crop strategy at 64×64 : resize the shorter edge to 64 pixels, then extract a 64×64 center crop, and store the result as `dogcat_train_64.npy` for training.

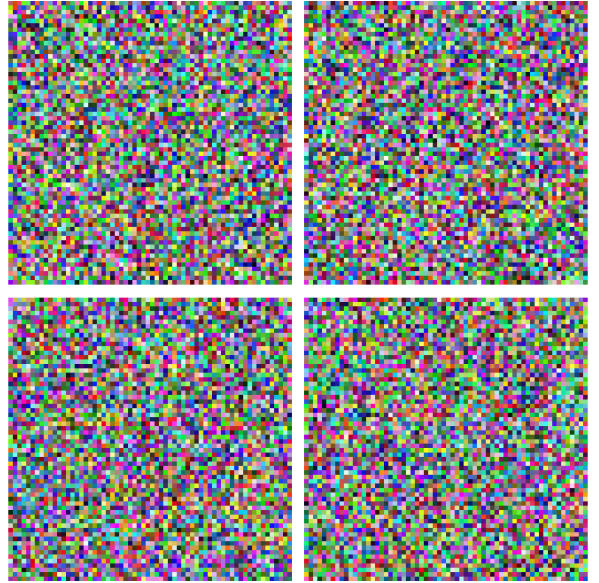


Figure 2: Sample images from the mixed "Dogs vs. Cats" dataset after scale-and-crop preprocessing (*chimera* pipeline uses 64×64 tensors).

2.2. Generative Model Architectures

Variational Autoencoders (VAE) VAEs define a probabilistic generative model via a learned latent variable distribution $q_\phi(z|x)$ parameterized by an encoder network. The gen-

erative process is specified by a decoder $p_\theta(x|z)$ where $z \sim \mathcal{N}(0, I)$. We evaluate two VAE variants:

β -VAE [6]: A standard VAE with convolutional encoder/decoder and a β parameter that controls the trade-off between reconstruction accuracy and prior matching. Grid search varies: `latent_dim` $\in \{64, 128\}$, $\beta \in \{1.0, 4.0\}$.

VQ-VAE-1: A discrete VAE using vector quantization in the latent space [7]. Instead of a continuous distribution, VQ-VAE discretizes the latent space into a learned codebook to reduce the blurriness typical of standard VAEs. Grid search varies: `codebook_size` $\in \{512, 1024\}$, `feat_map_size` $\in \{16 \times 16, 8 \times 8\}$, and reconstruction loss (L1 vs. MSE). Non-swept defaults are `embedding_dim` = 64 and `commitment_cost` = 0.25.

Generative Adversarial Networks (GAN) GANs formulate image generation as an adversarial game between a generator G and discriminator D . To directly address the mode collapse problem required by this study, we avoid the standard DCGAN and evaluate two stabilized variants:

WGAN-GP: Wasserstein GAN [8] with gradient penalty [9] uses the Wasserstein distance and adds a gradient penalty term to enforce the Lipschitz constraint, significantly reducing mode collapse and providing meaningful learning curves. Both GAN generators use a fixed latent dimension of 128. Grid search varies: `n_critc` $\in \{3, 5\}$, `batch_size` $\in \{64, 128\}$, `learning_rate_schedule` $\in \{\text{symmetric}, \text{TTUR}\}$.

SN-GAN: Spectral Normalization GAN [10] stabilizes discriminator training by normalizing weights with their largest singular value. Training uses one discriminator update per generator step and hinge losses. Grid search varies: `batch_size` $\in \{64, 128\}$, `learning_rate_schedule` $\in \{\text{symmetric}, \text{TTUR}\}$, `horizontal_flip_augmentation`.

Diffusion Models Diffusion models define a forward process that corrupts images to noise, then learn to reverse this process via a neural network. To mitigate the immense computational cost of diffusion, we focus on optimized architectures:

DDIM: A faster sampling approach for diffusion models [11] that uses a deterministic sampling procedure, enabling inference in fewer steps. A U-Net denoises 128×128 pixel images with an 8×8 bottleneck over $T=1000$ forward diffusion steps; an exponential moving average (EMA) copy of the U-Net is used for validation and sampling. Grid search varies: `noise_schedule` $\in \{\text{linear}, \text{cosine}\}$, `base_channels` $\in \{64, 128\}$, and `DDIM_steps` $\in \{50, 100\}$ (stored in each checkpoint and used at generation time).

Tiny Latent Diffusion Model (LDM): Following Rombach et al. [12], we train a lightweight U-Net diffusion model on *continuous* pre-quantization latents from a frozen VQ-VAE-1 encoder; vector quantization is applied only at decode time. Latent activations are scaled to unit variance using validation-set statistics before diffusion training. The VQ encoder/decoder weights are not updated during LDM training; only the U-Net (and its EMA copy) is optimized. DDIM sampling produces a latent tensor that is unscaled, quantized, and passed through the frozen decoder. The frozen VQ-VAE per seed is chosen from the VQ grid sweep as the checkpoint with lowest validation reconstruction loss (`prior_best_by_seed.json`). Grid search matches DDIM: `schedule` $\in \{\text{linear}, \text{cosine}\}$, `base_channels` $\in \{64, 128\}$, `DDIM_steps` $\in \{50, 100\}$.

PixelCNN Prior over VQ Codes (Baseline) VQ-VAE-1

alone defines $p(x|z)$ via the decoder but not a prior $p(z)$ over discrete code indices. For unconditional synthesis we therefore compare two learned priors on the same frozen VQ-VAE:

PixelCNN [13]: A lightweight autoregressive model with masked convolutions predicts codebook indices on the 16×16 or 8×8 latent grid (depending on the VQ-VAE configuration). Architecture is fixed at 128 hidden channels and 10 masked layers (not swept). After the VQ-VAE grid sweep, PixelCNN is trained once per seed on the manifest-selected frozen VQ-VAE; a follow-up `pixelcnn-experiment` script also trains a single Tiny LDM configuration per seed (EMA on, DDIM steps = 100) for wall-clock and qualitative comparison against PixelCNN.

2.3. Training Setup and Hyperparameter Grid

All models employ the Adam optimizer [14] with `lr` = 2×10^{-4} and $(\beta_1, \beta_2) = (0.5, 0.999)$ for VAE, diffusion, and PixelCNN trainers. GAN generators use $(\beta_1, \beta_2) = (0.5, 0.999)$ under WGAN-GP and $(0.0, 0.9)$ under SN-GAN. Training uses a high epoch cap (1000) with early stopping (`patience` = 15, `min_epochs` = 20, `min_delta` = 10^{-4}). β -VAE and VQ-VAE early stopping monitor **reconstruction loss** rather than total loss (which includes KL or VQ terms that can plateau early). PixelCNN and diffusion trainers monitor validation loss; GAN early stopping monitors generator loss (`g_loss`). SN-GAN horizontal-flip augmentation applies only to real images fed to the discriminator when enabled in the grid. All main experiments use Apple MPS unless overridden; metrics and sample grids are logged to MLflow.

Symmetric vs. TTUR learning-rate schedules [15] are implemented by pairing generator learning rate 2×10^{-4} with discriminator learning rate $\in \{2 \times 10^{-4}, 4 \times 10^{-4}\}$.

Each hyperparameter configuration is trained independently across three fixed random seeds {42, 0, 3407}. Seeds are reset in PyTorch, NumPy, and Python before every run. Grid-sweep results are compared per cell; evaluation metrics are aggregated over the same seed list for reproducibility. Table 1 lists the search space per family; the **Runs** column counts trained jobs as (grid cells) $\times$ 3 seeds. The main cat-face protocol comprises 135 such runs. The `pixelcnn-experiment` pipeline adds three fixed-configuration Tiny LDM trainings (DDIM steps = 100, EMA on) for prior wall-clock comparison against PixelCNN. The chimera extension adds three further WGAN-GP runs at 64×64 (141 training jobs in total).

Table 1: *Hyperparameter grid summary across generative model families.*

Model Family	Parameter	Search Space	Runs
β -VAE	<code>latent_dim</code> , β	$\{64, 128\} \times \{1.0, 4.0\}$	12
VQ-VAE-1	<code>codebook</code> , <code>map</code> , <code>loss</code>	$\{512, 1024\} \times \{16, 8\} \times \{L1, MSE\}$	24
WGAN-GP	<code>n_critc</code> , <code>batch</code> , <code>LR</code>	$\{3, 5\} \times \{64, 128\} \times \{\text{sym}, \text{TTUR}\}$	24
SN-GAN	<code>batch</code> , <code>LR</code> , <code>augment</code>	$\{64, 128\} \times \{\text{sym}, \text{TTUR}\} \times \{\text{no}, \text{hflip}\}$	24
DDIM	<code>schedule</code> , <code>ch.</code> , <code>steps</code> , <code>EMA</code>	$\{\text{lin}, \text{cos}\} \times \{64, 128\} \times \{50, 100\}$, EMA on	24
Tiny LDM	<code>schedule</code> , <code>ch.</code> , <code>steps</code> , <code>EMA</code>	same as DDIM (frozen VQ latents)	24
PixelCNN prior	<code>hidden width</code> , <code>depth</code>	128 channels, 10 masked layers (fixed)	3
Total (cat-face sweeps)			135

Includes 135 cat-face grid jobs; add +3 fixed Tiny LDM prior runs and +3 chimera jobs for 141 trainings total (see text).

2.4. Evaluation Metrics

Fréchet Inception Distance (FID) FID [15] compares the distribution of real validation images to unconditionally generated samples in the feature space of a pretrained Inception-v3 network. Lower FID indicates better alignment with the data distribution. We use each model’s native sampling procedure (e.g., $z \sim \mathcal{N}(0, I)$ for β -VAE and GANs, DDIM for pixel diffusion, random codes for VQ-VAE when no separate prior is evaluated). Generated tensors in $[-1, 1]$ are denormalized to $[0, 1]$, resized to 299×299 , and passed through Inception-v3 with ImageNet input normalization before feature extraction.

We report FID on the held-out validation set for every trained family: β -VAE, VQ-VAE-1, PixelCNN, Tiny LDM, WGAN-GP, SN-GAN, and DDIM, using a fixed sample count ($N=1000$) per run. For each model family we evaluate **every** hyperparameter grid cell that has a best checkpoint: FID is computed independently per seed, then mean $\pm$ standard deviation is reported per cell; the cell with the lowest mean FID is recorded as `best_run`. For VQ-linked models (VQ-VAE, PixelCNN, Tiny LDM), frozen VQ weights per seed come from `prior_best_by_seed.json` (lowest validation reconstruction across the VQ sweep). PixelCNN and Tiny LDM use their learned priors with that frozen decoder; VQ-VAE FID uses the trainer’s random-code debug sampler and is reported only as a weak baseline, not comparable to the learned priors.

Latent Interpolation Analysis For models with a continuous latent vector $z \in \mathbb{R}^d$, we test whether the learned representation is semantically smooth. We sample two endpoints $z_1, z_2 \sim \mathcal{N}(0, I)$ from the prior used at generation time, form eight uniformly spaced convex combinations $z_\alpha = (1 - \alpha)z_1 + \alpha z_2$, $\alpha \in \{0, \frac{1}{9}, \dots, 1\}$, and decode each z_α to an image, yielding a strip of ten frames. Continuity of appearance along the strip is judged qualitatively. This protocol is applied to β -VAE and WGAN-GP using the `best_run` hyperparameter cell from `fid_scores.json` (lowest mean FID); it is not used for discrete VQ codes or diffusion latents without an explicit interpolation definition.

PixelCNN vs. Tiny LDM Prior Comparison Using the same frozen VQ-VAE per seed, we compare autoregressive PixelCNN sampling against Tiny LDM (DDIM steps = 100, EMA) on wall-clock generation time and side-by-side sample grids. Timings use device synchronization on Apple MPS; outputs are written to `results/prior_comparison/`.

Exploratory Extension: Mixed Distributions (Chimera)

We train WGAN-GP on a mixed dataset containing both cats and dogs at 64×64 resolution ($\sim 25,000$ combined images, 90/10 train/val split per seed). The goal is to observe whether the unconditional model learns to separate the classes into mode-specific outputs or suffers from distribution averaging, resulting in generating “chimeras” (e.g., cat faces with dog ears or dog faces with feline features). This experiment probes the limits of unconditional generation under multi-modal input distributions. WGAN-GP was chosen as a stabilized adversarial baseline at reduced resolution for compute efficiency on the full mixed dataset. Diffusion models were too computationally expensive for $\sim 25,000$ images, and VAEs were rejected because blurred

reconstructions would obscure distribution averaging. Training uses the same early-stopping hyperparameters as the main cat study but defaults to a lower epoch cap (100) for this exploratory run; all three project seeds {42, 0, 3407} are trained independently.

3. Results

3.1. Fréchet Inception Distance

[To be populated from `results/fid_scores.json`.]

3.2. Qualitative Samples

[To be populated from `make_additional_samples` and training checkpoints.]

3.3. Latent Interpolation

[To be populated from `results/interpolations/`.]

3.4. PixelCNN vs. Tiny LDM

[To be populated from `results/prior_comparison/`.]

3.5. Chimera Extension

[To be populated from `checkpoints/chimera/`.]

4. Conclusions

[To be populated]

5. AI Disclosure

This report was edited with AI assistance, to improve clarity, consistency, and formatting. Part of the docstrings, some phrasing and code refactoring or debugging were refined using AI tools, but all scientific decisions and interpretations are the authors’ own. The core content, experimental design, and analysis were developed by the authors.

6. References

- [1] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [2] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf
- [3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf
- [4] D. Crawford, “Cats dataset,” <https://www.kaggle.com/datasets/crawford/cat-dataset>, 2018.
- [5] W. Cukierski, “Dogs vs. cats,” <https://kaggle.com/competitions/dogs-vs-cats>, 2013, kaggle.
- [6] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, “beta-VAE: Learning basic visual concepts with a constrained variational framework,” in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=Sy2fzU9gI>

- [7] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/7a98af17e63a0ac09ce2e96d03992fbc-Paper.pdf
- [8] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 214–223. [Online]. Available: <https://proceedings.mlr.press/v70/arjovsky17a.html>
- [9] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of wasserstein gans,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/892c3b1c6dcd52936e27cbd0ff683d6-Paper.pdf
- [10] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, “Spectral normalization for generative adversarial networks,” *CoRR*, vol. abs/1802.05957, 2018. [Online]. Available: <http://arxiv.org/abs/1802.05957>
- [11] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” *CoRR*, vol. abs/2010.02502, 2020. [Online]. Available: <https://arxiv.org/abs/2010.02502>
- [12] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 10 684–10 695.
- [13] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu, “Pixel recurrent neural networks,” in *International conference on machine learning*. PMLR, 2016, pp. 1747–1756.
- [14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [15] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a local nash equilibrium,” *Advances in neural information processing systems*, vol. 30, 2017.

A. Additional Experimental Details

End-to-end protocol. `make setup` → download/process cat data → `make run-all` (VAE, GAN, diffusion sweeps; VQ prior manifest; PixelCNN experiment) → `make eval` (FID + interpolations) → optional `make chimera` after `make process-dogcat-chimera`. Regenerate report figures with `make report-figures` after `make process-data`.

Pipeline order. `make run-all` executes VAE, GAN, and diffusion grid sweeps, writes the VQ-VAE prior manifest, then runs the PixelCNN experiment (train PixelCNN $\times 3$ seeds, train one Tiny LDM config per seed for prior comparison, wall-clock sample grids).

Logging. MLflow records hyperparameters, train/validation metrics, periodic 4×4 sample grids, and `samples.best.png` from the best checkpoint. Finished runs are not resumed on restart.

Prior comparison. PixelCNN vs. Tiny LDM generation time is measured with device synchronization on Apple MPS; outputs land in `results/prior.comparison/`.

Checkpoints. Best weights are saved per run as `best_seed{seed}.pt` inside `checkpoints/<model.type>/<slug>/`. Chimera training uses `checkpoints/chimera/`.

B. Code Availability

All code, configurations, and training logs are available at: <https://github.com/FlorekPawel/gen-cats>